

多集装箱堆场空间分配与车辆调度集成问题的建模与优化

李 坤¹, 唐立新², 陈树发³

(1. 天津工业大学 管理学院, 天津 300387; 2. 东北大学 物流优化与控制研究所, 沈阳 110004;

3. 天津工业大学 电气工程与自动化学院, 天津 300387)

摘 要 论文研究卸载集装箱堆场空间分配与车辆调度的集成作业问题. 在该问题中, 卸载集装箱动态到达, 车辆循环使用, 需要同时决策集装箱堆放位置及车辆的分配和路线. 对此问题建立整数规划数学模型, 考虑车辆调度约束以及堆场吊机操作等实际约束, 目标函数为最小化 makespan. 根据问题的特点设计两阶段禁忌搜索算法来求解此问题. 在实验中, 通过将禁忌搜索算法的结果同标准优化软件 CPLEX 所求得的最优解或下界比较, 算法求得了其中 7 组算例的最优解且剩余算例平均偏差小于 5%, 说明所设计的算法可以有效地解决该集成问题.

关键词 集装箱堆场; 车辆调度; Tabu search; makespan

Modeling and optimizing for the integrated problem with container storage allocation and truck scheduling

LI Kun¹, TANG Li-xin², CHEN Shu-fa³

(1. School of Management, Tianjin Polytechnic University, Tianjin 300387, China; 2. The Logistics Institute, Northeastern University, Shenyang 110004, China; 3. School of Electrical Engineering and Automation, Tianjin Polytechnic University, Tianjin 300387, China)

Abstract The integrated problem with truck scheduling and storage allocation was researched in this paper. Here, the container's arrival was dynamic and truck was used circularly, so the containers stacking position and truck scheduling had to be decided simultaneously. It was formulated as an integer programming model to minimize makespan of the whole discharging course. In the model, the truck scheduling constraints and stack crane operation constraints were also considered. Two stages tabu search algorithm was designed for solving above problem. The new method was compared with software CPLEX 11.0. The experiment demonstrates that the two stages tabu search obtains optimal solution for seven cases of total cases and the average gap of left cases is less than 5%. The experimental results show that this algorithm can solve this integrated problem effectively.

Keywords container storage; truck scheduling; Tabu search; makespan

1 引言

对集装箱码头来说, 通常每个码头终端包括码头作业区和集装箱存放区两大区域, 在码头作业区有一个或多个泊位供船舶在码头停靠, 并有多个岸吊来负责集装箱的装载和卸载. 集装箱堆场则用来临时堆放出港或进港的集装箱. 集装箱堆场通常由多个区 (block) 组成, 每个区有一个吊机负责本区内集装箱的相关移动. 场内卡车负责集装箱在码头作业区和堆放区之间的运输, 服务于堆场吊车和码头岸吊. 图 1 为典型的集装箱码头平面图.

集装箱码头关于分配和调度的问题, 例如泊位分配, 存储空间分配以及操作设备的调度等问题在过去几十年里已经有了很多研究, 但对于此类集成问题目前相关研究比较少. 岸吊主要用来装载和卸载停泊在泊位的船只. Daganzo^[1] 根据不同的岸吊调度策略设计了算法来确定指派给多船泊位的岸吊数量. Kim^[2] 把岸吊

收稿日期: 2011-12-12

资助项目: 国家自然科学基金 (71032004)

作者简介: 李坤 (1981-), 男, 讲师, 研究方向: 生产计划与调度优化, 交通物流优化; 唐立新 (1966-), 男, 教授, 博士生导师, 研究方向: 生产与运输中的调度理论, 智能优化方法和调度系统的开发.

调度问题作为 m 个并行机调度问题来研究并建立了以最小集装箱操作完成时间为目标函数的混合整数规划模型. Lee 等^[3-4] 建立了带有岸吊操作约束的混合整数规划模型并应用遗传算法对问题进行了求解.

对于堆场的各种操作来说, 堆场吊机的调度问题是一类重要的研究问题, 吊机调度的优劣直接影响到堆场的作业效率. Zhang 等^[5] 对动态吊机调度问题建立了混合整数规划模型并应用拉格朗日松弛对问题进行了求解. Kim^[6] 对进口集装箱的最优存储空间数量以及吊机数量设计了费用模型. Ng^[7] 研究了一类多吊机调度问题, 该问题的特点是只有一个双向移动轨道的堆场中的集装箱带有不同的准备时间. 场内卡车通常负责集装箱在码头操作区和堆场之间的运输. Vis^[8] 基于最大流问题设计了启发式用以确定 AGVs (automated guided vehicle) 的车队规模. Kim 等^[9-10] 对 straddle carrier 的最优路径问题建立了模型并设计了相应的算法. Nishimura 等^[11] 提出了动态拖车指派问题并设计了启发式算法来求解此问题.

集装箱堆场空间分配问题和车辆调度问题已吸引了众多学者的研究, 且两问题之间有较强的关联性. Bish 等^[12] 首次将两问题作为一集成问题来研究, 作者假设每个集装箱对应多个存放位置, 集装箱由船舶卸载后由堆场卡车负责运送到集装箱堆场, 每辆卡车一次只能运送一单位集装箱. 对于该集成问题的求解, Bish 设计了两阶段启发式算法并将该集成问题分解为两个独立的子问题分别求解. 算法第一阶段在不考虑车辆调度的基础上确定集装箱的堆场位置, 第二阶段求解已分配位置集装箱的车辆调度问题. Lee, Cao, Shi^[13] 等学者针对堆场车辆调度和集装箱堆场空间分配集成问题也进行了研究. 该集成问题的目标为最小化加权总延误和车辆总的运输时间, 设计了混合插入算法求解集成问题, 通过对该问题中关键参数等方面的实验表明了算法的有效性.

本论文对该集成问题的研究与以上学者研究的区别主要有: 1) 求解该集成问题的思路与 Bish 等学者的研究有所不同. 本论文研究的集成问题第一阶段为车辆调度问题, 第二阶段为集装箱堆场空间分配问题. 针对这两个阶段分别应用禁忌搜索算法进行求解, 并将第二阶段求解的结果反馈到第一阶段以调整更新卡车调度方案, 而文献^[12] 没有更多考虑两个子问题之间的相互关系. 2) 与文献^[13] 虽研究的同为集装箱堆场空间分配和车辆调度集成问题, 但对该问题特征的研究角度多有不同. 文献^[13] 中所研究的问题可以归结为 VRPTW 问题, 并且集成研究出口、进口集装箱问题, 本文只针对进口集装箱的堆场空间分配和车辆调度集成问题进行研究.

本文对集装箱堆场空间分配问题和车辆调度问题进行了集成, 根据集成问题的特点以最小化集装箱的最大操作完成时间为目标函数建立了整数规划模型并基于该问题的实际操作约束提出了一个两阶段禁忌搜索算法 (two-phase tabu search, 简称 TPTS) 来获得该问题的近优解.

2 数学模型

针对该问题的特点, 建立便于描述问题的整数规划模型, 目标函数为最小化集装箱的最大操作完成时间, 具体模型如下:

1) 参数

i, j : 集装箱索引符号.

k : 场内卡车索引符号.

l : 堆场中区的索引符号.

m : 卸载集装箱的数量.

n : 场内卡车数量.

b : 堆场中堆放集装箱的区的数量.

r_i : 岸吊对集装箱 i 的允许操作时间.

t_i : 场内卡车运输集装箱 i 的开始时间.

λ_l : 从码头操作区到堆场中 l 区的运输时间.

δ : 堆场吊机从场内卡车卸载集装箱到某区内的时间.

d_i : 场内卡车运输集装箱 i 的运输时间.

$\tilde{t}_i$: 堆场吊机从场内卡车卸载集装箱的开始时间.

C_i : 集装箱 i 被放置到堆场区内的完成时间.

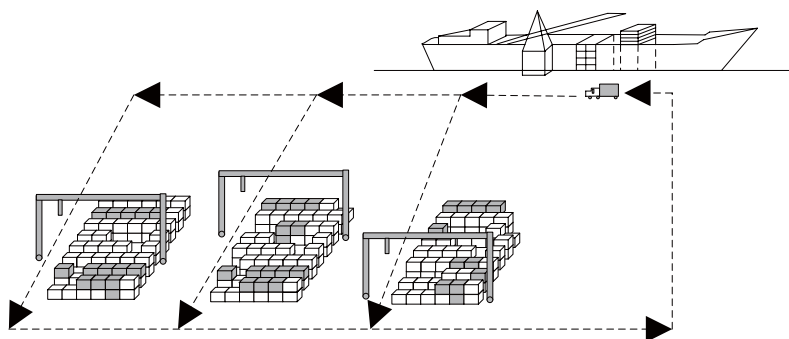


图 1 典型的集装箱码头平面图

M : 足够大的正数.

2) 决策变量

$$x_{ijk} = \begin{cases} 1 & \text{如果卡车 } k \text{ 运输完集装箱 } i \text{ 运输集装箱 } j \\ 0 & \text{否则} \end{cases} \quad k = 1, 2, \dots, n; j = 1, 2, \dots, m; i \neq j \quad (1)$$

$$y_{ijl} = \begin{cases} 1 & \text{如果集装箱 } i \text{ 在集装箱 } j \text{ 紧前被堆放在堆场 } l \text{ 区} \\ 0 & \text{否则} \end{cases} \quad l = 1, 2, \dots, b; i, j = 1, 2, \dots, m \quad (2)$$

$$u_{ik} = \begin{cases} 1 & \text{如果堆场卡车 } k \text{ 运输集装箱 } i \\ 0 & \text{否则} \end{cases} \quad k = 1, 2, \dots, n; j = 1, 2, \dots, m \quad (3)$$

$$v_{il} = \begin{cases} 1 & \text{如果集装箱被分配到堆场中的 } l \text{ 区} \\ 0 & \text{否则} \end{cases} \quad l = 1, 2, \dots, b; i = 1, 2, \dots, m \quad (4)$$

3) 数学模型

$$\min \max_{i=1,2,\dots,m} C_i \quad (5)$$

$$C_i = \tilde{t}_i + \delta, \quad i = 1, 2, \dots, m \quad (6)$$

$$d_i = \sum_{l=1}^b v_{il} \lambda_l, \quad i = 1, 2, \dots, m \quad (7)$$

$$t_i \geq r_i, \quad i = 1, 2, \dots, m \quad (8)$$

$$\tilde{t}_i \geq t_i + d_i, \quad i = 1, 2, \dots, m \quad (9)$$

$$t_i + d_i \leq M(1 - y_{ijl}) + t_j + d_j, \quad i, j = 1, 2, \dots, m, i \neq j, l = 1, 2, \dots, b \quad (10)$$

$$\tilde{t}_i + \delta \leq M(1 - y_{ijl}) + \tilde{t}_j, \quad i, j = 1, 2, \dots, m, i \neq j, l = 1, 2, \dots, b \quad (11)$$

$$\tilde{t}_i - d_i \leq M(1 - x_{ijk}) + t_j, \quad i, j = 1, 2, \dots, m, i \neq j, k = 1, 2, \dots, n \quad (12)$$

$$\sum_{k=1}^n u_{ik} = 1, \quad i = 1, 2, \dots, m \quad (13)$$

$$\sum_{l=1}^b v_{il} = 1, \quad i = 1, 2, \dots, m \quad (14)$$

$$x_{ijk} \leq u_{ik}, \quad i, j = 1, 2, \dots, m, i \neq j, k = 1, 2, \dots, n \quad (15)$$

$$x_{ijk} \leq u_{jk}, \quad i, j = 1, 2, \dots, m, i \neq j, k = 1, 2, \dots, n \quad (16)$$

$$y_{ijl} \leq v_{il}, \quad i, j = 1, 2, \dots, m, i \neq j, l = 1, 2, \dots, b \quad (17)$$

$$y_{ijl} \leq v_{jl}, \quad i, j = 1, 2, \dots, m, i \neq j, l = 1, 2, \dots, b \quad (18)$$

$$x_{ijk} + x_{jik} \leq 1, \quad i, j = 1, 2, \dots, m, i \neq j, k = 1, 2, \dots, n \quad (19)$$

$$x_{ijk} + x_{jik} \geq u_{ik} + u_{jk} - 1, \quad i, j = 1, 2, \dots, m, i \neq j, k = 1, 2, \dots, n \quad (20)$$

$$y_{ijl} + y_{jil} \leq 1, \quad i, j = 1, 2, \dots, m, i \neq j, l = 1, 2, \dots, b \quad (21)$$

$$y_{ijl} + y_{jil} \geq v_{il} + v_{jl} - 1, \quad i, j = 1, 2, \dots, m, i \neq j, l = 1, 2, \dots, b \quad (22)$$

$$u_{ik}, v_{il}, x_{ijk}, y_{ijl} \in \{0, 1\}, \quad i, j = 1, 2, \dots, m, l = 1, 2, \dots, b, k = 1, 2, \dots, n \quad (23)$$

$$C_i, d_i, t_i, \tilde{t}_i \in Z^+, \quad i = 1, 2, \dots, m \quad (24)$$

目标函数 (5) 是最小化集装箱的最大操作完成时间. 约束 (6) 是计算每个集装箱完成时间的等式. 约束 (7) 表示计算每个集装箱的运输时间. 约束 (8) 表示每个集装箱从码头操作区被运输到堆场区的时间约束. 约束 (9) 表示每个集装箱被堆场吊机从卡车卸载到堆场区内的时间约束. 约束 (10) 强制变量, (11) 表示集装箱被运输到堆场后遵守先到先卸载的规则. 约束 (12) 表示堆场卡车只有在运输完前一个集装箱后返回码头

操作区才能运输下一个集装箱. 约束 (13) 和 (14) 表示每个集装箱只能由一辆卡车来运输并只能被放到一个堆场内. 从约束 (15) 到 (18) 表示各个决策变量之间的相互关系. 约束 (19) 和 (20) 保证如果集装箱 i 和集装箱 j 由同一辆卡车 k 来运输, 则卡车 k 先运输集装箱 i 后运输集装箱 j 或先运输集装箱 j 后运输集装箱 i . 约束 (17) 和 (18) 和约束 (19)、(20) 类似, 表示的是堆场区的吊机操作. 约束 (23) 和 (24) 表示决策变量的范围. 对此集成问题很难在允许的时间范围内求得其最优解, 因此在下面章节中设计了贪婪启发式求其初始解并以此解为基础通过禁忌搜索来改进从而获得近优解.

3 初始解的贪婪启发式算法

作为禁忌搜索的基础解, 初始解的质量很大程度上影响了搜索的时间以及最终解的质量. 对本章中研究的集成问题设计了基于贪婪思想的启发式, 其主要算法主要思想为对在码头作业区的卸载集装箱按其可操作时间依次分配给可最先使用的卡车, 并将卡车运输的集装箱分配给最先卸载到堆场的区. 以下为启发式算法的主要步骤.

Step 1 定义 p_k 为卡车 k 的空闲时间, 其中 $k = 1, 2, \dots, n$, q_l 为堆场 l 区吊机的空闲时间, 其中 $l = 1, 2, \dots, b$. 设 $p_k = 0$, $q_l = 0$, $i = 1$.

Step 2 分配最小空闲时间的卡车 k' 运输集装箱 i , 更新 $t_i = \max\{p_{k'}, r_i\}$.

Step 3 计算 $l' = \max\{t_i + \lambda_l, q_l\}$, 并分配集装箱 i 到 l' 区, 更新 $\tilde{t}_i = \max\{t_i + \lambda_{l'}, q_{l'}\}$, $p_{k'} = \tilde{t}_i + \lambda_{l'}$, $q_{l'} = \tilde{t}_i + \delta$, $C_i = \tilde{t}_i + \delta$.

Step 4 判断 i 值和 n 值, 如果 $i = n$, 转到下一步, 否则更新 $i = i + 1$, 转到 Step 2.

Step 5 计算目标函数值 $\max_{i=1,2,\dots,m} C_i$.

4 两阶段禁忌搜索算法 (TPTS)

禁忌搜索算法是 Glover^[14] 和 Hansen^[15] 在 1986 年提出的并已经被证明对求解企业中存在的复杂组合优化问题十分有效. 不同于爬山邻域搜索算法, 禁忌搜索算法不容易陷入局部最优, 因为禁忌搜索算法允许移动从当前解获得一个劣解以期在以后迭代中获得更好的解. 目前, 禁忌搜索被大量应用于组合优化问题, 并且得以不断改进, 其中 Tuzun 设计了求解配送中心选址 - 车辆路径问题的两阶段禁忌搜索算法, 其算法的思想有效反映不同决策层之间的相互作用关系^[16]. 本文研究的集成问题可以划分为两个阶段, 第一阶段为车辆调度问题, 第二阶段为集装箱堆场空间分配问题. 针对这两个阶段分别应用禁忌搜索算法进行求解. 在卡车调度阶段, 通过禁忌搜索以获得一个较好的卡车调度方案, 并以此调度方案为基础在集装箱堆放空间分配阶段再次应用禁忌搜索. 在第二阶段中每执行一次 move 移动则反馈到第一阶段以更新相应的卡车调度方案, 因而可以说第一阶段的禁忌搜索可以看做整个算法的 local search 过程.

4.1 解的表达形式

为了清晰描述解的邻域结构提出了解的向量表示法. 定义 S 为一个完整的调度集合, S_{j1} 表示集装箱在第一阶段由堆场卡车 j 的运输集合, S_{l2} 表示集装箱在第二阶段由堆场 l 区的吊机的处理集合, 且 $S = S_{j1} \cup S_{l2}$, 其中 $j = 1, 2, \dots, n$, $l = 1, 2, \dots, b$. 设 $n_{j1} = |S_{j1}|$, $b_{l2} = |S_{l2}|$, S_{j1} 和 S_{l2} 可以定义为以下两个置换向量:

$$\pi_{j1} = (\pi_{j1}(1), \pi_{j1}(2), \dots, \pi_{j1}(n_{j1})) \quad (25)$$

$$\pi_{l2} = (\pi_{l2}(1), \pi_{l2}(2), \dots, \pi_{l2}(n_{l2})) \quad (26)$$

其中 $\pi_{j1}(x)$ 表示 π_{j1} 中的第 x 个集装箱, $\pi_{l2}(x)$ 表示 π_{l2} 中的第 x 个集装箱, 那么 $\pi_1 = (\pi_{11}, \pi_{21}, \dots, \pi_{n1})$ 可以用来表示卡车在第一阶段的调度方案, $\pi_2 = (\pi_{12}, \pi_{22}, \dots, \pi_{b2})$ 可以表示集装箱在第二阶段的堆场分配方案, 因而该集成问题完整的解可以表示如下:

$$\pi = \pi_1 \cup \pi_2 = (\pi_{11}, \pi_{21}, \dots, \pi_{n1}) \cup (\pi_{12}, \pi_{22}, \dots, \pi_{b2}) \quad (27)$$

4.2 车辆调度阶段禁忌搜索

由于在两阶段禁忌搜索算法中, 第一阶段中应用的禁忌搜索算法被当做整个算法的邻域搜索过程, 相当于整个算法的一个嵌入算法, 因而此搜索过程的运行时间直接决定了整个算法的运行时间, 因而在第一阶段的禁忌搜索中只应用了 swap move 一种类型的移动方式来产生邻域.

Swap move: 交换两集装箱所分配的卡车. 向量 $\pi_{j_1 1} = (\pi_{j_1 1}(1), \pi_{j_1 1}(2), \dots, \pi_{j_1 1}(u), \dots, \pi_{j_1 1}(n_{j_1 1}))$ 中的元素 $\pi_{j_1 1}(u)$ 表示集装箱 i_1 由卡车 j_1 负责运输, 向量 $\pi_{j_2 1} = (\pi_{j_2 1}(1), \pi_{j_2 1}(2), \dots, \pi_{j_2 1}(v), \dots, \pi_{j_2 1}(n_{j_2 1}))$ 中的元素 $\pi_{j_2 1}(v)$ 表示集装箱 i_2 由卡车 j_2 运输, 其中 $j_1, j_2 = 1, 2, \dots, n$ 且 $j_1 \neq j_2$. 当执行 swap 移动后, 两向量相应地变为:

$$\pi_{j_1 1} = (\pi_{j_1 1}(1), \pi_{j_1 1}(2), \dots, i_2, \dots, \pi_{j_1 1}(n_{j_1 1})) \quad (28)$$

$$\pi_{j_2 1} = (\pi_{j_2 1}(1), \pi_{j_2 1}(2), \dots, i_1, \dots, \pi_{j_2 1}(n_{j_2 1})) \quad (29)$$

需要指出的是, 交换所涉及到的集装箱 i_1 和 i_2 原堆场分配方案不发生变化.

停止准则: 以判断当前解不改进迭代次数是否达到预设值 (Max_1) 为第一阶段禁忌搜索算法的停止准则以期在有效时间内得到所需卡车分配方案, 并以此为基础开始第二阶段禁忌搜索.

4.3 堆场空间分配阶段禁忌搜索

在第二阶段的禁忌搜索中采用了两种移动方式产生邻域, $N_1(R)$ 和 $N_2(R)$. 当在 $N_1(R)$ 邻域内找到一个最好的非禁忌 move 时, 跳出当前禁忌搜索算法, 转入第一阶段禁忌搜索得到相应的最好车辆分配方案并更新解, 进而继续执行当前 (第二阶段) 禁忌搜索算法对 $N_1(R)$ 邻域进行搜索, 直到满足停止准则更新解并转入 $N_2(R)$ 邻域进行搜索; 同样的, 当在 $N_2(R)$ 邻域内找到一个最好的非禁忌 move 时, 跳出第二阶段禁忌搜索算法, 并执行第一阶段禁忌搜索得到相应的最好车辆分配方案并更新解, 进而继续执行第二阶段禁忌搜索算法对 $N_2(R)$ 邻域进行搜索, 直至满足停止准则, 更新解, 算法终止. 在第二阶段禁忌搜索中 $N_1(R)$ 邻域对应的移动方式为 swap move, $N_2(R)$ 邻域对应的移动方式为 shift move, 下面分别介绍这两种移动方式.

1) swap move: 交换两卡车所运输集装箱的目的地, 即 $\pi_{l_1 2} = (\pi_{l_1 2}(1), \pi_{l_1 2}(2), \dots, \pi_{l_1 2}(u), \dots, \pi_{l_1 2}(n_{l_1 2}))$ 中的元素 $\pi_{l_1 2}(u)$ 和向量 $\pi_{l_2 2} = (\pi_{l_2 2}(1), \pi_{l_2 2}(2), \dots, \pi_{l_2 2}(v), \dots, \pi_{l_2 2}(n_{l_2 2}))$ 中的元素 $\pi_{l_2 2}(v)$, $l_1 \neq l_2$.

2) shift move: 重新分配卡车所运输集装箱到另一堆场区. 在此种移动中也是同时改变解的两个向量, 如将向量 $\pi_{l_1 2} = (\pi_{l_1 2}(1), \pi_{l_1 2}(2), \dots, \pi_{l_1 2}(u), \dots, \pi_{l_1 2}(n_{l_1 2}))$ 中的元素 $\pi_{l_1 2}(u)$ 删除后添加到向量 $\pi_{l_2 2} = (\pi_{l_2 2}(1), \pi_{l_2 2}(2), \dots, \pi_{l_2 2}(v), \dots, \pi_{l_2 2}(n_{l_2 2}))$ 中, 其中 $l_1 \neq l_2$.

停止准则: 在第二阶段禁忌搜索算法中, 对 swap 邻域的搜索和对 shift 邻域的搜索是串联进行的, 因而对两个搜索分别设定了最大不改进迭代次数, 当达到第一个预设值 (Max_2) 时, swap 邻域搜索结束, 更新解向量后进行 shift 邻域搜索, 当迭代达到第二个预设值 (Max_3) 时, shift 邻域搜索也结束, 得到最终解向量, 整个算法终止.

4.4 加速策略

4.4.1 加速策略 1

为了更好地说明加速策略 1, 给出了如下的性质.

性质 1 在卡车调度阶段, 交换前 n 个集装箱所分配的卡车不能使当前解得到任何改进.

证明 卡车在初始状态下全部可使用, 由于存在岸吊对集装箱的可操作时间, 即每个集装箱都有开始时间, 所以任意分配卡车给前 n 个集装箱对目标函数不会产生影响, 因而交换前 n 个集装箱所分配的卡车得到的新解与原解的目标函数值相同.

4.4.2 加速策略 2

基于约束 (8), 如果算法第一阶段中的 swap 移动以及第二阶段中的 swap 和 shift 移动导致新的解向量违反约束 (8), 那么就放弃这个移动.

4.5 算法流程图

两阶段禁忌搜索的流程如图 2 所示.

5 实验结果

为了验证所提出的模型和算法的有效性, 使用 C 语言实现了算法并在 Pentium4 3.0G CPU, 1024MB 内存的个人计算机上进行了运算.

5.1 实验参数及数据

与其他智能优化算法一样, 两阶段禁忌搜索算法中的参数的变化对解也产生很大的影响. 算法中的参数主要有禁忌表的长度以及最大不改进迭代次数. 通过实验数据测试, 发现当禁忌表的长度为 10 时所得的结

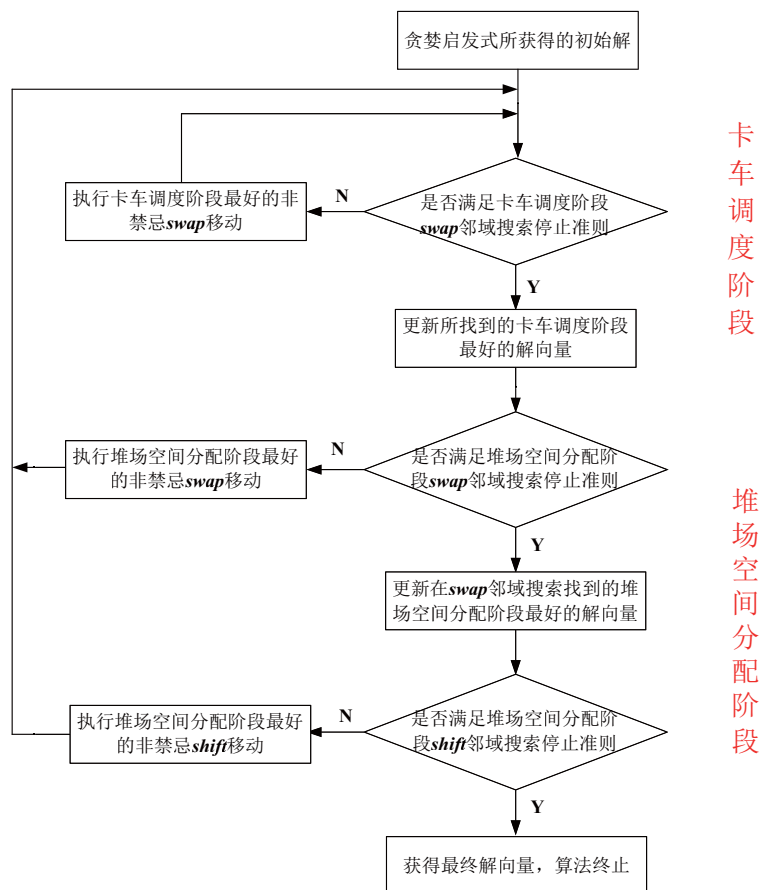


图 2 两阶段禁忌搜索流程图

果比较好, 因而在接下来的实验中, 禁忌表的长度统一设置为 10. 该算法中的三个最大不改进迭代次数在实验中取值 $\text{Max_1}=30$, $\text{Max_2}=\text{Max_3}=50$.

为了测试所提出的算法的性能, 本研究使用随机产生的实例对算法进行了测试. 实验中总共用到了 20 个规模的算例, 每个算例随机生成了 10 组数据, 具体数据的产生如下: 进口集装箱的开始时间在 $[0, 50]$ 范围内均匀分布, 卡车从码头到堆场每个区的运输时间在 $[12, 60]$ 范围内均匀分布, 堆场吊机从场内卡车卸载集装箱到某区内的时间为定值 10. 表 1 给出了 20 个规模算例的信息, 包括集装箱数量, 卡车数量以及堆场中区的数量.

5.2 实验结果

在表 2 中给出了 20 个规模算例分别使用启发式、两阶段禁忌搜索以及 CPLEX11.0 软件的运算结果. 其中 Obj 列表示三种方法分别获得的目标函数值. 对于 CPLEX 软件设置最大运行时间为 60 分钟, 当软件消耗的计算时间达到 60 分钟时, 即使软件没有达到最优解, 也将停止计算. 如果优化软件在时间限制内得到最优解, 则其 gap 值为 0; 否则得到的当前最好解与下界之间的平均偏差在表中给出. 根据 CPLEX 提供的下界, 可以计算出两阶段禁忌搜索算法得到的解与下界之间的平均偏差.

基于表 2 中所提供的实验结果, 可以得到如下结论:

- 1) 所设计的贪婪启发式算法对于小规模测试集合可以得到令人满意的解, 例如从实验算例 1 到算例 7, 贪婪启发式算法可以获得该 7 个算例的最优解, 并且算例 8 到算例 15 的平均偏差小于 5%.
- 2) 两阶段禁忌搜索算法对于算例 1 到算例 7 同样可以获得最优解, 对其它算例获得了比贪婪启发式更好的解. 显然, 对于求解大规模此集成问题两阶段禁忌搜索比贪婪启发式具有更大的优势.
- 3) CPLEX11.0 软件所求得解要优于贪婪启发式, 对于算例 1 到算例 10, 软件都获得了最优解. 与两阶段禁忌搜索算法比较, 对于求解大规模算例两阶段禁忌搜索算法所求得解要优于软件, TPTS 求解 20 个算例的平均偏差为 5.99%, 软件求解 20 个算例的平均偏差为 6.85%. 在此需要说明的是, 如果在软件所设置最大求解时间足够大的情况下, 可能获得更好的解. 比较 TPTS 算法与 CPLEX11.0 的计算时间, TPTS 算法的计算时间明显优于优化软件, TPTS 算法求解 20 个算例的平均计算时间仅有 36.7 秒.

4) 在算例 16 到算例 18 的三个算例中可以看出两阶段禁忌搜索算法在求解算例时, 在 m 值相等的条件下, 计算时间随着 n 值和 b 值的增加而增加。

表 1 实验算例配置

算例	m	n	b
1	5	2	3
2	10	2	3
3	10	3	3
4	15	3	3
5	15	4	3
6	20	4	3
7	20	5	3
8	30	5	3
9	30	6	3
10	30	5	4
11	35	7	3
12	35	7	4
13	40	8	3
14	40	9	4
15	40	8	5
16	45	8	3
17	45	9	4
18	45	9	5
19	50	10	3
20	50	10	5

表 2 CPLEX 11.0 与 TPTS 算法结果比较

算例	启发式			TPTS			CPLEX			
	Obj	Gap(%)	Time(s)	Obj	Gap(%)	Time(s)	Obj	LB	Gap(%)	Time(s)
1	117.8	0	0.13	117.8	0	0.50	117.8	117.8	0	2.11
2	232.1	0	0.58	232.1	0	2.37	232.1	232.1	0	4.52
3	207.6	0	0.52	207.6	0	2.01	207.6	207.6	0	4.48
4	281.8	0	0.47	281.8	0	1.75	281.8	281.8	0	7.89
5	263.9	0	0.51	263.9	0	1.82	263.9	263.9	0	7.68
6	324.5	0	0.78	324.5	0	2.30	324.5	324.5	0	14.51
7	315.7	0	0.92	315.7	0	3.30	315.7	315.7	0	14.77
8	573.4	0.47	1.47	573.4	0.47	7.26	570.7	570.7	0	147.61
9	561.4	0.63	2.43	561.4	0.63	9.49	557.9	557.9	0	155.77
10	568.8	2.78	2.01	568.8	2.78	8.51	553.4	553.4	0	157.4
11	630.1	3.94	5.24	630.1	3.94	17.21	637.7	606.2	5.20	1260
12	628.7	2.68	5.73	628.7	2.68	19.34	634.5	612.3	3.63	1278
13	789.4	4.78	5.23	774.2	2.76	25.14	780.4	753.4	3.58	1354
14	780	2.69	5.74	779.8	2.66	32.14	777.1	759.6	2.3	1424
15	766.8	2.23	6.76	760.1	1.33	36.26	763.3	750.1	1.76	1362
16	924.3	10.38	15.78	902.7	7.8	32.21	916.7	837.4	9.47	1524
17	903	10.58	18.42	887	8.62	37.7	891.4	816.6	9.16	1526
18	887.6	10.43	16.2	849.6	5.71	39.11	876.3	803.7	9.03	1783
19	1109.4	15.84	28.17	1054.8	10.14	46.1	1080	957.7	12.78	3600
20	1173.4	14.7	22.43	1129.7	10.43	42.49	1141.6	1023	11.59	3600

参考文献

- [1] Daganzo C F. Crane productivity and ship delay in ports[J]. Transportation Research Record, 1989(1251): 1-9.
- [2] Kim K H. A crane scheduling method for port container terminals[J]. European Journal of Operational Research, 2004, 156: 752-768.
- [3] Lee D H, Wang H Q, Miao L X. Quay crane scheduling with non-interference constraints in port container terminals[J]. Transportation Research Part E, 2008, 44(1): 124-135.
- [4] Lee D H, Cao Z, Meng Q. Scheduling of two-transtainer systems for loading outbound containers in port container terminals with simulated annealing algorithm[J]. International Journal of Production Economics, 2007, 107(1): 115-124.
- [5] Zhang C Q, Liu J Y, Wan Y W. Storage space allocation in container terminals[J]. Transportation Research Part B, 2003, 37: 883-903.
- [6] Kim K H, Kim H B. The optimal sizing of the storage space and handling facilities for import containers[J]. Transportation Research Part B, 2002, 36: 821-835.
- [7] Ng W C. Crane scheduling in container yards with inter-crane interference[J]. European Journal of Operational Research, 2005, 164: 64-78.
- [8] Vis I F A. Minimum vehicle fleet size under time-window constraints at a container terminal[J]. Transportation Science, 2005, 39(2): 249-260.
- [9] Kim K Y, Kim K H. A routing algorithm for a single straddle carrier to load export containers onto a container-ship[J]. International Journal of Production Economics, 1999, 59: 425-433.
- [10] Kim K H, Kim K Y. Routing straddle carriers for the loading operation of containers using a beam search algorithm[J]. Computers & Industrial Engineering, 1999, 36: 109-113.
- [11] Nishimura E, Imai A, Papadimitriou S. Yard trailer routing at a maritime container terminal[J]. Transportation Research Part E, 2005, 41: 53-76.
- [12] Bish E K, Leong T Y, Li C L, et al. Analysis of a new vehicle scheduling and location problem[J]. Naval Research Logistics, 2001, 48: 363-385.
- [13] Lee D H, Cao J X, Shi Q X, et al. A heuristic algorithm for yard truck scheduling and storage allocation problems[J]. Transportation Research Part E, 2009, 45: 810-820.
- [14] Glover F. Tabu search — Part I[J]. ORSA Journal on Computing, 1989, 1(3): 190-206.
- [15] Hansen P. The steepest ascent mildest descent heuristic for combinatorial programming[C]// Congress on Numerical Methods in Combinatorial Optimization, Capri, Italy, 1986.
- [16] Tuzun D, Burke L I. A two-phase tabu search approach to the location routing problem[J]. European Journal of Operational Research, 1999, 116(1): 87-99.